

Real-time Facial and Body Keypoint Analysis for Interactive Video Applications

Project Code: SWS3026_08 | Group Members: Zhang Zonghao · Wang Xiaorui · Li Yunzang · Zhang Yunxiang

Abstract

A laptop-first keypoint pipeline powers three live webcam demos: expression effects, reaction-lag-aware dance scoring and gesture-controlled gameplay. The design prioritises real-time response and explainable geometry.

Main Contributions

- Shared face/body keypoint pipeline.
- Expression: 46.33% Macro-F1; 18.08 ms.
- Scale-, mirror- and reaction-lag-aware pose scoring.

Research question

Can one lightweight keypoint pipeline support several live demos on an ordinary laptop?

1. Facial Landmark Detection

Each webcam frame is processed by Haar/YuNet face detection followed by 68-point LBF landmarks. Models load once; the display reports the face box, landmarks and FPS.

Method

- Webcam: OpenCV capture.
- Face: Haar cascade; YuNet alternative.
- Landmarks: LBF, 68 points.
- Robustness: CLAHE/gamma, ROI tracking and smoothing.

Robustness observations

Condition	Observed behaviour
Low light	CLAHE restores local contrast.
Head rotation	Haar/LBF degrade off-frontal.
Occlusion	Covered features distort landmarks.
Fast motion	Blur and jitter need smoothing.

Practical lesson. A missed face box cannot be recovered by the landmark stage.

2. Why Use Keypoints?

Normalised landmark geometry replaces raw pixels, reducing input size and background bias while keeping prediction below the 30 ms target.

Feature group	Dimension / role
Normalised x/y	136 values (68 × x/y)
Geometry features	38 distances, ratios and regional shape cues
Total input	174 keypoint-derived features

Discussion and Limitations

- Pose, blur and occlusion remain difficult.
- Landmark geometry omits texture.
- Partial bodies weaken movement scoring.

From a camera frame to something you can play with

Our project in one glance — face landmarks, expression, pose matching, and interaction

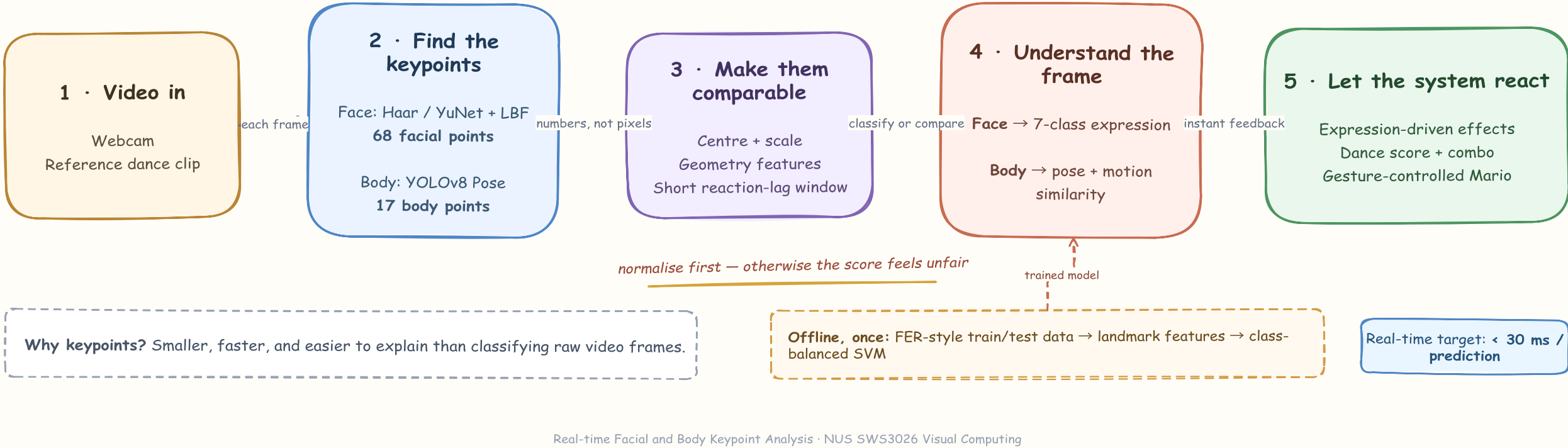


Fig. 1. Webcam or reference video is converted into sparse keypoints, normalised, analysed and mapped to an interactive response.

3. Expression Recognition

Landmarks are extracted from the official FER-style train/test split. Model selection is restricted to the training split; the official test split is used once for final evaluation.

Training and model selection

- Seven classes; 174 keypoint-derived features.
- PCA benchmarked as a controlled comparison.
- Selection prioritises Macro-F1, class balance and latency.
- Final model: class-balanced RBF-SVM.

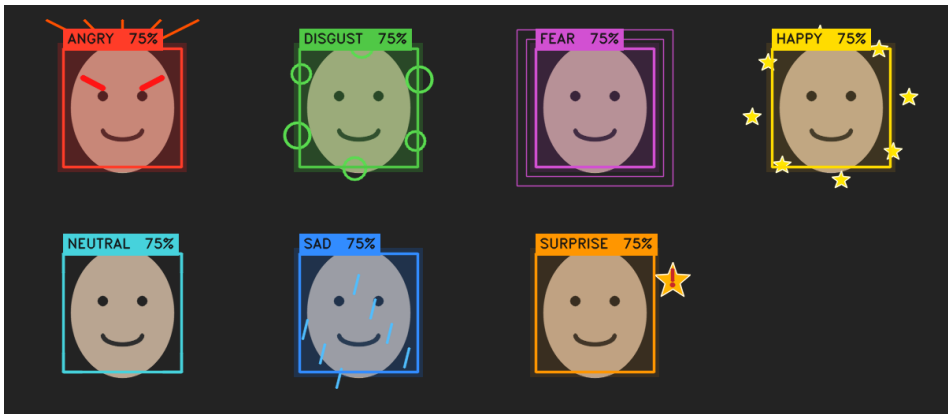


Fig. 2. Seven live effects with label and confidence.

47.31%	46.33%	18.08 ms
test accuracy	Macro-F1	prediction time

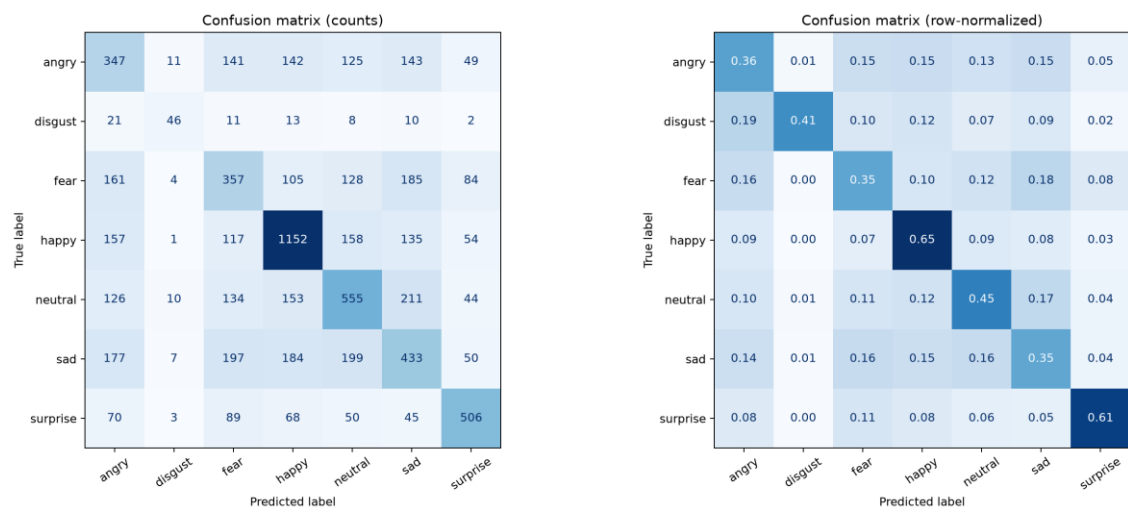


Fig. 3. Final confusion matrices; visually similar classes remain difficult to separate.

Failure analysis



Fig. 4. Typical errors under ambiguity, low contrast, pose and occlusion.

Interpretation. Minority and visually similar classes remain the main weakness.

Conclusion and Future Work

A keypoint-centred design supports three live applications on a laptop. Next steps are Stratified K-fold confirmation, stronger tracking and a multi-user latency study.

4. Pose-based Applications

YOLOv8n Pose provides 17 COCO keypoints for the reference dancer and webcam participant. Metrics below use a 2,680-frame CPU reference run at imgsz=320.

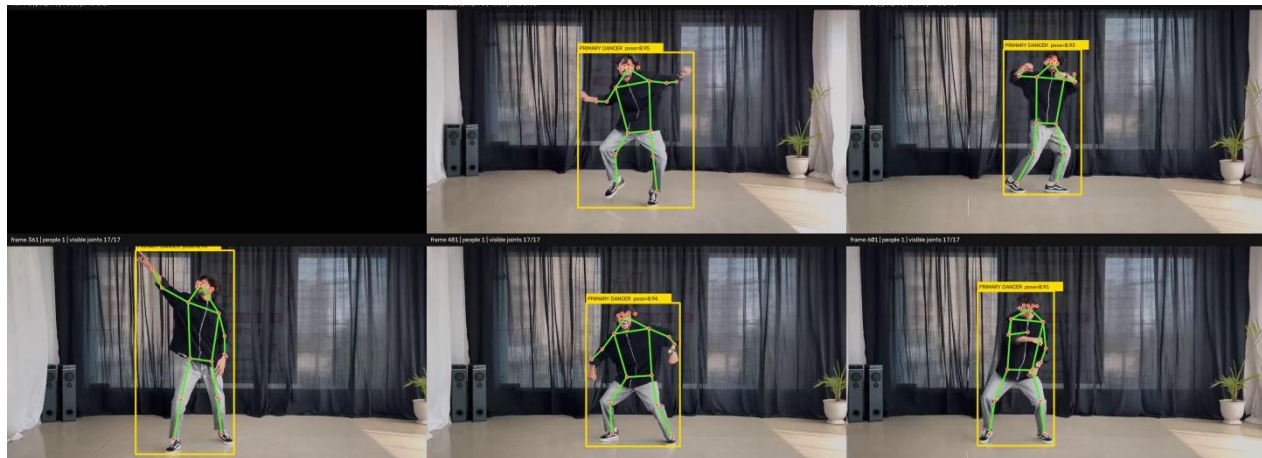


Fig. 5. CPU reference run with temporal primary-dancer tracking.

91.16%	31.40 ms	23.09 FPS
dancer detected	pose inference	processing rate

Spatial and temporal alignment

- Spatial: centre and scale by the torso.
- Temporal: search a short reaction-lag window.
- Similarity: combine pose geometry and motion.
- Mirror: compare reflected left/right joints.

$$\text{Active score} = 0.55 \times \text{Pose} + 0.45 \times \text{Motion}$$

Feedback design

Feedback uses PERFECT / GREAT / GOOD / MISS, plus score and combo. HOLD and MOVE states prevent static poses from earning repeated points.

5. Gesture-controlled Game

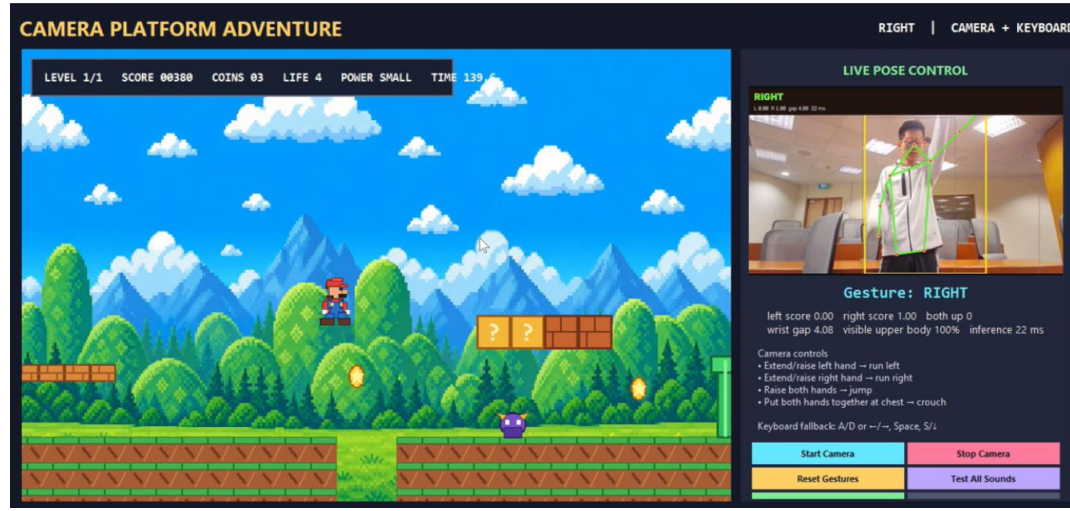


Fig. 6. The same pose stream controls a platform-game demonstration.

Gesture mapping: lean or move → left/right; raise the body → jump; lower the body → crouch.

Practical lesson. Scale normalisation and lag handling reduce sensitivity to body size and reaction time.

Project and Acknowledgements

NUS School of Computing Summer Workshop (SWS3026). Thanks to the teaching team and contributors.

GitHub Repository · github.com/yunli2024/26Summer_NUS_SOC_Visual_Computing_Project_G8